

Contents

1 Chats	1
1.1  Gemini	1

1 Chats

Exported on: 3/17/2026, 12:19:11 PM

1.1 Gemini

Show thinking

Gemini said

NeuralSQL: Relational Database as a Differentiable Computational Engine

The implementation of a differentiable computational graph entirely within a relational database management system (RDBMS) requires mapping standard tensor operations to relational algebra. By leveraging PostgreSQL's PL/pgSQL procedural language and standard SQL JOIN constraints, we can execute forward and backward passes, manage state, and compute gradients without relying on external array-processing libraries or languages.

Below is the technical breakdown of the NeuralSQL architecture, demonstrating how a Multi-Layer Perceptron (MLP) is constructed and optimized.

1. Relational Representation of Sparse Tensors

Traditional deep learning frameworks allocate contiguous blocks of memory for tensors. NeuralSQL utilizes a sparse representation distributed across normalized tables, which inherently optimizes for zero-valued elements typical in functions like ReLU or zero-padded data.

Tensor metadata and discrete non-zero elements are decoupled:

Vectors: Defined in `sparse_vectors` (tracking vector dimensionality). Non-zero elements are stored in `sparse_vector_elements` with explicit integer bounds CHECK (index >= 0) and CHECK (index < size).

Matrices: Defined in `sparse_matrices` (tracking row/column dimensions). Elements reside in `sparse_matrix_elements` mapped by (row, col) tuples.

This schema guarantees that memory scales with the informational density of the network rather than its absolute parameter count.

2. Linear Algebra via Relational Algebra

The core operation of a dense neural network layer is the affine transformation:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

In NeuralSQL, matrix-vector multiplication is executed as an inner join combined with an aggregate grouping. The computational graph maps the column index of the weight matrix $\mathbf{W}$ to the element index of the input vector $\mathbf{x}$:

```

1 INSERT INTO sparse_vector_elements (vector_id, index, value)
2 SELECT result_id, m.row AS index, SUM(m.value * v.value) AS value
3 FROM sparse_matrix_elements m
4 JOIN sparse_vector_elements v ON m.col = v.index
5 WHERE m.matrix_id = $1 AND v.vector_id = $2
6 GROUP BY m.row
7 HAVING ABS(SUM(m.value * v.value)) > 1e-15;

```

Proof of Operation: The JOIN isolates pairs where W_{ij} and x_j are both non-zero. The SUM aggregate computes the dot product for each row i , yielding the pre-activation vector $\mathbf{z}$.

3. State Caching and Forward Propagation

To compute gradients during backpropagation, intermediate tensor states must be preserved.

The `forward_dense` function coordinates the forward pass for a specific layer. It executes the affine transformation and maps a non-linear activation function (e.g., ReLU, Sigmoid) over the resulting vector. Crucially, it caches the inputs and pre-activations into the `forward_cache` table, using `(layer_id, sample_id)` as a unique composite key. This acts as the differentiable state tape.

4. Backpropagation and the Chain Rule

The `backward_dense` function computes parameter gradients using the cached states. Given an upstream gradient from the subsequent layer or loss function ($\nabla_{\mathbf{a}} L$), the system applies the chain rule.

Activation Gradient: Computes the derivative of the activation function with respect to the pre-activation $\mathbf{z}$. For ReLU, this operates sparsely by projecting 1.0 where $\mathbf{z} > 0$.

Local Error (δ): Computes the Hadamard product ($\odot$) of the upstream gradient and the activation gradient via an INNER JOIN on matching indices.

$$\delta = \nabla_{\mathbf{a}} L \odot \sigma'(\mathbf{z})$$

Weight Gradient: Computes the outer product $\delta \mathbf{x}^T$ via a CROSS JOIN between the δ vector elements and the cached input vector $\mathbf{x}$ elements.

Input Gradient: Propagates the error backwards ($\mathbf{W}^T \delta$) using a transposed matrix-vector multiplication (grouping by `m.col` instead of `m.row`).

5. Gradient Accumulation and Optimization

Mini-batch gradient descent requires accumulating gradients across multiple independent samples before updating the weights. NeuralSQL manages this statefully via the `gradient_accumulators` table.

The `batch_training_step` procedure iterates over a designated array of sample IDs, accumulating the computed gradients. Once the batch completes, `apply_gradient_descent` executes the parameter update:

$$\mathbf{W}_{new} = \mathbf{W}_{old} - \frac{\alpha}{N} \sum_{k=1}^N \nabla_{\mathbf{W}_k} L$$

This is implemented by scaling the accumulated gradient matrix by $-\frac{\alpha}{N}$ and executing a FULL OUTER JOIN to add it to the existing weight matrix.

6. Pipeline Orchestration via CLI

Bash scripts provide the execution environment, replacing high-level frameworks like Python.

Ingestion: The script `import_mnist_gawk.sh` utilizes `gawk` to stream CSV text processing, generating optimized SQL INSERT statements into a staging table, circumventing memory bottlenecks during data load.

Execution: `neuralsql.sh` acts as the command dispatcher, wrapping database calls (`psql`) into standard arguments (e.g., `init`, `create-network`, `train`).

Telemetry: Training metrics (loss, accuracy) are stored immutably in `epoch_metrics`. A lightweight Bun-based TypeScript server (`server.ts`) proxies a PostgREST API to expose these metrics to an HTML frontend, leveraging Plotly for real-time visualization.

Would you like me to draft a Bun script that automates the generation of dynamic SQL queries for hyperparameter grid search across this database architecture?

You stopped this response
